VISVESVARAYA TECHNOLOGICAL UNIVERSITY

BELAGAVI - 590 018, KARNATAKA

A Report on

"IMAGE PROCESSING: RGB-TO-GRAYSCALE CONVERSION USING CUDA"

(BCS702)

In

INFORMATION SCIENCE AND ENGINEERING

By

Ms. Kshama S Hiremath **USN: 1BY23IS097**

Mr. Vinay Kumar Y B
Assistant Professor
Department of ISE, BMSIT&M.

BMS Institute of Technology and Management
(An Autonomous Institution Affiliated to VTU, Belagavi)
Avalahalli, Doddaballapur Main Road, Bengaluru – 560119

2026-2027

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1	Abstract	3
2	Introduction	4
3	Problem Statement	5
4	Objectives	5
5	Background / Theoretical Foundation	6
6	CUDA Programming Model	8
7	RGB-to-Grayscale Conversion Case Study	10
8	Implementation	12
9	Experimental Setup	20
10	Results and Performance Analysis	23
11	Advantages of CUDA-Based RGB-to-Grayscale Conversion	26
12	Limitations	27
13	Applications of RGB-to-Grayscale Conversion	27
14	Conclusion	28

1. Abstract

Image processing is an important application of parallel computing, particularly when large amounts of image data must be processed efficiently. This case study focuses on converting colour images into grayscale images using NVIDIA CUDA (Compute Unified Device Architecture). In this process, each pixel's Red, Green, and Blue (RGB) components are converted into a single grayscale intensity value using a weighted formula.

The implementation is developed using CUDA C++, where each GPU thread processes one pixel independently. A sequential CPU implementation is also developed to provide a reference for correctness verification and performance comparison. The program is tested with input sizes of 10,000, 100,000, 1,000,000, and 10,000,000 pixels using an NVIDIA Tesla T4 GPU in Google Colab.

The experimental results show that GPU execution is slower for smaller workloads due to kernel-launch and memory-transfer overhead. For larger workloads, GPU parallelism provides better performance. In the recorded experiment, the GPU achieved an end-to-end speedup of 1.87× for 10 million pixels. All tested workloads reported zero mismatched pixels when comparing CPU and GPU outputs within the specified tolerance. This study demonstrates the application of GPU-based parallel processing to image-processing operations.

Keywords:- CUDA, Parallel Computing, GPU, RGB-to-Grayscale Conversion, Image Processing, CUDA C++, CPU-GPU Performance Comparison, NVIDIA Tesla T4.

2. Introduction

Modern computational workloads increasingly rely on parallel processing to achieve efficient performance when handling large datasets and computationally intensive operations. Graphics Processing Units (GPUs), originally designed for graphics rendering, have evolved into powerful parallel processors capable of executing thousands of threads concurrently. NVIDIA's Compute Unified Device Architecture (CUDA) provides a programming model that enables developers to utilize GPU parallelism through extensions to the C and C++ programming languages.

RGB-to-grayscale conversion is a fundamental image processing operation that transforms a colour image into a grayscale image by converting the Red (R), Green (G), and Blue (B) components of each pixel into a single intensity value. This operation is well suited for CUDA because each pixel can be processed independently, allowing the workload to be distributed across multiple GPU threads. The conversion demonstrates important concepts of GPU programming, including memory allocation on the host and device, data transfer, kernel execution, synchronization, and result retrieval.

This report presents an implementation of RGB-to-grayscale conversion using CUDA C++, developed and executed on an NVIDIA Tesla T4 GPU through the Google Colab platform. The performance of the CUDA implementation is compared with a sequential CPU implementation across different input sizes to evaluate execution time, speedup, and output correctness.

3. Problem Statement

For a pixel at position ((x,y)), the grayscale intensity can be calculated from its Red (R), Green (G), and Blue (B) components using the weighted average formula:

$$Gray(x, y) = 0.299R + 0.587G + 0.114B$$

In a sequential implementation, this computation is performed pixel by pixel on a CPU. As the number of pixels increases, processing large images can require more execution time. Since the grayscale value of each pixel can be calculated independently, the problem is inherently parallelizable.

The objective of this case study is to implement RGB-to-grayscale conversion on a GPU using CUDA so that multiple pixels can be processed simultaneously, and to evaluate the resulting performance relative to a sequential CPU implementation.

4. Objectives

1. To implement RGB-to-grayscale image conversion using CUDA C++.
2. To utilize GPU parallelism by assigning individual CUDA threads to process pixels simultaneously.
3. To develop a sequential CPU implementation for comparison with the CUDA-based implementation.
4. To measure and compare CPU and GPU execution times for different input sizes.
5. To verify output correctness and evaluate the performance improvement achieved through CUDA parallel processing.

5. Background / Theoretical Foundation

5.1 Parallel Computing

Parallel computing is an approach in which multiple computational operations are performed simultaneously rather than being handled one after another. A large problem can be divided into smaller tasks and distributed among several processing units, allowing these tasks to execute concurrently. This approach is particularly useful in image processing, where operations on different pixels can often be performed independently. Since the grayscale conversion of one pixel does not depend on the result of another pixel, the computation can be distributed across multiple processing units, making parallel computing well suited for accelerating RGB-to-grayscale conversion.

5.2 GPU Computing

A Graphics Processing Unit (GPU) is a highly parallel processor containing a large number of processing cores capable of handling numerous data elements concurrently. Its architecture follows the Single Instruction, Multiple Threads (SIMT) model, allowing multiple threads to execute the same operation on different data elements. Unlike a CPU, which focuses on executing a limited number of complex tasks with low latency, a GPU is designed to achieve high throughput by processing many similar operations simultaneously. This architecture makes GPUs particularly effective for image processing, where multiple pixels can be converted from RGB to grayscale in parallel.

5.3 CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA for performing general-purpose computations on NVIDIA GPUs. It provides extensions to C and C++ that enable developers to create functions known as *kernels*, which are executed by multiple GPU threads simultaneously. In RGB-to-grayscale conversion, each CUDA thread processes one pixel by reading its red, green, and blue components, applying the grayscale conversion formula, and storing the resulting intensity value. This approach allows a large number of pixels to be processed concurrently, improving computational efficiency for sufficiently large workloads.

5.4 CPU vs GPU

The CPU and GPU use different architectural approaches and are suited to different types of computational workloads. A CPU generally contains a smaller number of powerful cores designed to handle sequential operations, complex instructions, and tasks requiring low response time. In contrast, a GPU contains many processing cores that can perform similar operations on numerous data elements concurrently. For RGB-to-grayscale conversion, a CPU can process pixels sequentially, whereas a GPU can distribute pixel calculations among multiple threads. However, GPU performance also depends on the input size and the overhead of transferring data between CPU and GPU memory.

Aspect	**CPU**	**GPU**
Number of cores	Relatively few powerful cores	Many processing cores
Core complexity	High; designed for complex operations	Designed for high parallel throughput
Optimisation goal	Low latency	High throughput
Best suited for	Sequential and control-flow-intensive tasks	Data-parallel operations
Processing approach	Can process pixels sequentially	Can process multiple pixels concurrently
Memory considerations	Uses system memory and CPU caches	Uses GPU memory with high bandwidth
Application in this study	Sequential grayscale conversion	Parallel grayscale conversion using CUDA

Table 1. Comparison of CPU and GPU architectural characteristics.

6. CUDA Programming Model

6.1 Host and Device

In CUDA terminology, the **host** represents the CPU along with the main system memory, while the **device** represents the GPU and its dedicated memory. In RGB-to-grayscale conversion, the host manages the input RGB pixel data, allocates the required resources, and controls program execution. The input data is transferred from the host to the device, where CUDA kernels perform grayscale conversion using multiple threads. After processing is completed, the resulting grayscale data is transferred back from the device to the host for verification and further use.

6.2 Threads

A thread is the basic unit of execution in the CUDA programming model. Each thread executes the same kernel instructions while working on a specific portion of the input data. In RGB-to-grayscale conversion, each CUDA thread is assigned to process one pixel by reading its Red, Green, and Blue components, applying the grayscale conversion formula, and storing the resulting intensity value. This allows multiple pixels to be converted concurrently.

6.3 Blocks

Threads in CUDA are arranged into groups known as **thread blocks**, which organize parallel execution. Each block contains multiple threads that process different pixels of the input data. Threads within the same block can cooperate and share data through shared memory when required. In this case study, blocks organize the threads responsible for converting different RGB pixels into grayscale values.

6.4 Grid

A **grid** represents the complete collection of thread blocks created for a single CUDA kernel execution. It defines the overall organization of the parallel workload by specifying the number of blocks required to process the input data. In RGB-to-grayscale conversion, the grid contains enough blocks and threads to cover the input pixels, enabling the GPU to distribute the conversion operations across multiple threads.

6.5 Global Thread Index

Since a CUDA kernel executes across multiple blocks and threads, each thread must identify the specific pixel it is responsible for processing. A unique global index is calculated by combining the block position with the thread position inside that block. For a one-dimensional arrangement, the index is calculated as:

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

where **blockIdx.x** represents the position of the current block in the grid, **blockDim.x** indicates the number of threads in each block, and **threadIdx.x** specifies the position of the current thread within its block.

In this case study, the calculated index identifies the pixel to be processed. Each valid thread uses this index to access the corresponding RGB components, compute the grayscale intensity, and store the result. A boundary check ensures that threads do not access pixels beyond the input size.

7. RGB-to-Grayscale Conversion Case Study

7.1 Problem Definition

The case study requires converting every pixel in an input colour image from RGB format into grayscale format. Each output pixel is calculated using the weighted sum of its Red (R), Green (G), and Blue (B) components:

[Gray = 0.299R + 0.587G + 0.114B]

The objective is to implement this operation using both a sequential CPU approach and a parallel CUDA approach, and compare their execution times for different input sizes.

7.2 Sequential Approach

In a sequential CPU implementation, a loop processes all pixels one after another and calculates the grayscale intensity of each pixel using its RGB components.

For each pixel, the CPU reads the Red, Green, and Blue values, applies the grayscale conversion formula, and stores the resulting intensity value.

This approach processes pixels sequentially, so the execution time generally increases as the number of pixels grows. Since each pixel can be converted independently, the operation can also be parallelized using CUDA to utilize the GPU's processing capabilities.

7.3 Algorithm

SEQUENTIAL CPU ALGORITHM:

Sequential CPU Algorithm (RGB to Grayscale) C / C++

```
1  // RGB to Grayscale Conversion (CPU - Sequential)
2  void rgbToGrayscaleCPU(unsigned char *input,
3                         unsigned char *output,
4                         int width, int height) {
5    int numPixels = width * height;
6
7    for (int i = 0; i < numPixels; i++) {
8       int idx = i * 3;                // RGB index
9       unsigned char R = input[idx];
10      unsigned char G = input[idx + 1];
11      unsigned char B = input[idx + 2];
12
13      unsigned char gray = (unsigned char)(0.299f * R + 0.587f * G + 0.114f * B);
14      output[i] = gray;               // Grayscale value
15   }
16 }
```

CUDA PARALLEL ALGORITHM:

CUDA Parallel Algorithm (RGB to Grayscale) CUDA C++

```
1  // RGB to Grayscale Conversion (CUDA Kernel)
2   __global__ void rgbToGrayscaleCUDA(
3              unsigned char *input,    // RGB input image (3 channels)
4              unsigned char *output,   // Grayscale output image (1 channel)
5              int width, int height) {
6      int x = blockIdx.x * blockDim.x + threadIdx.x;
7      int y = blockIdx.y * blockDim.y + threadIdx.y;
8      if (x < width && y < height) {
9        int idx = (y * width + x) * 3;     // RGB index
10       unsigned char R = input[idx];
11       unsigned char G = input[idx + 1];
12       unsigned char B = input[idx + 2];
13       unsigned char gray = (unsigned char)(0.299f * R + 0.587f * G + 0.114f * B);
14       int outIdx = y * width + x;        // Grayscale index
15       output[outIdx] = gray;
16     }
17   }
18
```

7.4 CUDA Execution Pipeline

1. **Memory Allocation:** Allocate CPU host memory using malloc and GPU device memory using cudaMalloc.
2. **Data Transfer:** Copy RGB input data from the CPU to the GPU using cudaMemcpy with cudaMemcpyHostToDevice.
3. **Kernel Execution:** Launch the CUDA kernel using the <<<grid, block>>> execution configuration syntax. Each thread processes one pixel and computes its grayscale intensity.
4. **Result Retrieval:** Copy the grayscale output from the GPU to the CPU using cudaMemcpy with cudaMemcpyDeviceToHost.
5. **Memory Deallocation:** Release CPU memory using free and GPU memory using cudaFree.

This five-stage pipeline forms the structural foundation of the implementation described in Section 8.

8. Implementation

The complete CUDA C++ source code developed and executed for this case study is reproduced below:

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include <chrono>
#include <iostream>
#include <iomanip>

using namespace std;
using namespace chrono;

// Sequential CPU implementation
void rgbToGrayscaleCPU(unsigned char *input,
                       unsigned char *output, int N)
{
   for (int i = 0; i < N; i++)
   {
      int idx = i * 3;

      unsigned char R = input[idx];
      unsigned char G = input[idx + 1];
      unsigned char B = input[idx + 2];

      output[i] = (unsigned char)(
         0.299f * R + 0.587f * G + 0.114f * B);
   }
}

// CUDA parallel kernel
```

```
__global__ void rgbToGrayscaleCUDA(unsigned char *input,
                                   unsigned char *output, int N)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < N)
    {
        int idx = i * 3;

        unsigned char R = input[idx];
        unsigned char G = input[idx + 1];
        unsigned char B = input[idx + 2];

        output[i] = (unsigned char)(
            0.299f * R + 0.587f * G + 0.114f * B);
    }
}

int main()
{
    int sizes[] = {10000, 100000, 1000000, 10000000};
    int numSizes = sizeof(sizes) / sizeof(sizes[0]);

    cout << "RGB TO GRAYSCALE CONVERSION USING CUDA\n";

    for (int s = 0; s < numSizes; s++)
    {
        int N = sizes[s];

        size_t rgbBytes = (size_t)N * 3;
        size_t grayBytes = (size_t)N;
```

```
// Host memory allocation
unsigned char *h_input =
    (unsigned char *)malloc(rgbBytes);
unsigned char *h_cpu =
    (unsigned char *)malloc(grayBytes);
unsigned char *h_gpu =
    (unsigned char *)malloc(grayBytes);

if (!h_input || !h_cpu || !h_gpu)
{
    cout << "Host memory allocation failed.\n";
    free(h_input);
    free(h_cpu);
    free(h_gpu);
    return 1;
}

// Initialize RGB pixel data
for (int i = 0; i < N; i++)
{
    h_input[3 * i]     = (i * 17) % 256;
    h_input[3 * i + 1] = (i * 31) % 256;
    h_input[3 * i + 2] = (i * 47) % 256;
}

// CPU execution time
auto cpuStart = high_resolution_clock::now();

rgbToGrayscaleCPU(h_input, h_cpu, N);

auto cpuEnd = high_resolution_clock::now();
```

```
double cpuTime =
    duration<double, milli>(cpuEnd - cpuStart).count();

// Device memory allocation
unsigned char *d_input, *d_output;

cudaMalloc((void **)&d_input, rgbBytes);
cudaMalloc((void **)&d_output, grayBytes);

// Start GPU timing (including memory transfers)
auto gpuStart = high_resolution_clock::now();

// Transfer RGB input from host to device
cudaMemcpy(d_input, h_input, rgbBytes,
           cudaMemcpyHostToDevice);

// Configure CUDA grid and blocks
int threadsPerBlock = 256;
int blocksPerGrid =
    (N + threadsPerBlock - 1) / threadsPerBlock;

// Launch CUDA kernel
rgbToGrayscaleCUDA<<<blocksPerGrid, threadsPerBlock>>>(
    d_input, d_output, N);

cudaDeviceSynchronize();

// Transfer grayscale output to host
cudaMemcpy(h_gpu, d_output, grayBytes,
           cudaMemcpyDeviceToHost);

auto gpuEnd = high_resolution_clock::now();
```

```
    double gpuTime =
        duration<double, milli>(gpuEnd - gpuStart).count();

    // Verify CPU and GPU results
    int mismatches = 0;

    for (int i = 0; i < N; i++)
    {
        if (h_cpu[i] != h_gpu[i])
            mismatches++;
    }

    double speedup = cpuTime / gpuTime;

    cout << fixed << setprecision(4);
    cout << "\nInput pixels: " << N << endl;
    cout << "CPU time: " << cpuTime << " ms\n";
    cout << "GPU total time: " << gpuTime << " ms\n";
    cout << "Speedup: " << speedup << "x\n";
    cout << "Mismatches: " << mismatches << endl;
    cout << "Correctness: "
         << (mismatches == 0 ? "PASS" : "FAIL") << endl;

    // Release device and host memory
    cudaFree(d_input);
    cudaFree(d_output);

    free(h_input);
    free(h_cpu);
    free(h_gpu);
}
```

```
    cout << "\nExecution completed successfully.\n";

    return 0;
}
```

8.1 Image and Host Memory Allocation

Host memory for the input RGB data h_input and output grayscale data h_cpu and h_gpu is allocated using the standard C malloc function. The input buffer is sized to store three colour components (Red, Green, and Blue) for each pixel, while each output buffer stores one grayscale intensity value per pixel. The input RGB data is initialized with sample pixel values, and the output buffers are reserved to store the results generated by the CPU and CUDA implementations.

8.2 Device Memory Allocation

Corresponding device buffers d_input and d_output are allocated on the GPU using cudaMalloc. These buffers are stored in GPU global memory and are accessible to CUDA threads during kernel execution. The d_input buffer holds the original RGB pixel data, while d_output stores the grayscale values produced by the CUDA kernel.

8.3 Host-to-Device Transfer

The input RGB data stored in h_input is transferred from host memory to GPU memory using cudaMemcpy with the cudaMemcpyHostToDevice direction flag. This transfer makes the original pixel data available in the device buffer d_input, allowing the CUDA kernel to access and process the RGB components during parallel execution.

8.4 CUDA Kernel Launch

The rgbToGrayscaleCUDA kernel is launched using the CUDA execution configuration syntax <<<blocksPerGrid, threadsPerBlock>>>. In this implementation, each block contains 256 threads, and the number of blocks is calculated based on the total number of pixels. Each CUDA thread calculates its global index and processes one pixel by reading its RGB components, applying the weighted grayscale conversion formula, and storing the resulting intensity value. A boundary check ensures that threads do not process pixels beyond the input size.

8.5 Device-to-Host Transfer

After the grayscale conversion kernel finishes execution, cudaDeviceSynchronize() is used to ensure that GPU operations have completed. The processed grayscale data stored in d_output is then transferred back to the host buffer h_gpu using cudaMemcpy with the cudaMemcpyDeviceToHost direction flag. The returned data can then be compared with the CPU-generated output.

8.6 Memory Deallocation

Once the image processing operation is completed, the memory allocated on the GPU is released using cudaFree. The corresponding host memory buffers are released using free. Releasing these resources prevents unnecessary memory retention and helps avoid memory leaks during repeated executions.

8.7 Correctness Verification

The correctness of the grayscale output is verified by comparing the pixel values generated by the CUDA implementation with those produced by the sequential CPU implementation. The program examines each output pixel and counts any mismatches. If the mismatch count is zero, the results are considered correct. This verification confirms that the CUDA kernel produces the same grayscale output as the CPU implementation for the tested input data.

9. Experimental Setup

9.1 Platform

The CUDA C/C++ implementation was developed and tested using **Google Colab**, a cloud-based environment that provides access to NVIDIA GPU resources. The platform was used to compile and execute the CUDA program and evaluate the performance of RGB-to-grayscale conversion using both sequential CPU processing and parallel GPU processing.

9.2 Hardware

The experiment was performed using the GPU resource provided by Google Colab. The CUDA program was executed on an **NVIDIA Tesla T4 GPU**, which provides dedicated device memory and parallel processing capabilities for handling large workloads. The GPU was used to process RGB pixel data concurrently and generate the corresponding grayscale output.

Component	Specification
GPU	NVIDIA Tesla T4
GPU memory	Approximately 15 GB
Platform	Google Colab
Processing architecture	GPU-based parallel processing
Programming environment	C++ with NVIDIA CUDA

Table 2. Hardware specifications of the experimental environment.

9.3 Software

The software environment was configured to support the development and execution of the CUDA C/C++ program. The NVIDIA CUDA Toolkit provides the nvcc compiler required to compile the CUDA source code, while the NVIDIA driver enables communication between the application and the GPU.

The program was written in C++ with CUDA extensions and compiled using nvcc. The compiler version used during the experiment was CUDA compilation tools, release 13.0, V13.0.88.

Item	Value
Programming language	C++ with NVIDIA CUDA

Compiler	nvcc
CUDA compiler version	Release 13.0, V13.0.88
Execution platform	Google Colab
GPU runtime	NVIDIA CUDA

Table 3. Software and compiler configuration.

9.4 CUDA Configuration

The implementation was evaluated using four input sizes: 10,000, 100,000, 1,000,000, and 10,000,000 pixels. Each RGB pixel contains three colour components, while the grayscale output contains one intensity value per pixel.

The CUDA kernel uses a one-dimensional grid configuration with **256 threads per block**. The number of blocks is calculated according to the total number of pixels to ensure that sufficient threads are launched to process the complete input.

Parameter	**Value**
Input sizes (N)	10,000; 100,000; 1,000,000; 10,000,000
Threads per block	256
Blocks per grid	((N + 255) / 256), using integer division
Global thread index	blockIdx.x * blockDim.x + threadIdx.x
Boundary condition	if (i < N)
Grayscale formula	(Gray = 0.299R + 0.587G + 0.114B)

Table 4. CUDA kernel launch configuration.

9.5 Benchmark Methodology

The performance study compares the sequential CPU implementation with the corresponding CUDA GPU implementation using four different input sizes. For each input size, the CPU execution time and the total GPU execution time are measured and compared to evaluate the effect of workload size on performance.

The GPU measurement represents the complete processing workflow rather than kernel execution alone. Therefore, the recorded time includes:

- Host-to-device transfer of the RGB input data.
- CUDA grayscale conversion kernel execution.

- Device synchronization.
- Device-to-host transfer of the grayscale output.

The CPU implementation measures the time required to perform grayscale conversion sequentially. The GPU speedup is calculated using the following formula:

[\text{Speedup} =\frac{\text{CPU Execution Time}}{\text{GPU Total Execution Time}}]

A speedup greater than 1 indicates that the measured GPU workflow was faster than the CPU implementation for that input size, whereas a value below 1 indicates that the GPU workflow took longer. The CPU and GPU outputs are also compared pixel by pixel to verify correctness. This methodology evaluates both computational performance and the overhead associated with transferring data between CPU and GPU memory.

10. Results and Performance Analysis

10.1 Benchmark Results

The measured execution times for the sequential CPU and CUDA GPU implementations, along with the resulting end-to-end speedup, are presented in Table 5. Four input sizes were tested to evaluate the effect of workload size on RGB-to-grayscale conversion performance.

Input Pixels (N)	CPU Time (ms)	GPU Total Time (ms)	End-to-End Speedup	Result
10,000	0.0569	0.2060	0.28×	Correct
100,000	0.3690	0.2869	1.29×	Correct
1,000,000	2.7814	1.6536	1.68×	Correct
10,000,000	25.2972	13.5599	1.87×	Correct

Table 5. End-to-end CPU vs GPU benchmark results.

10.2 CPU vs GPU Comparison

Figure 5 presents the measured CPU and GPU execution times across the four tested input sizes for RGB-to-grayscale conversion.

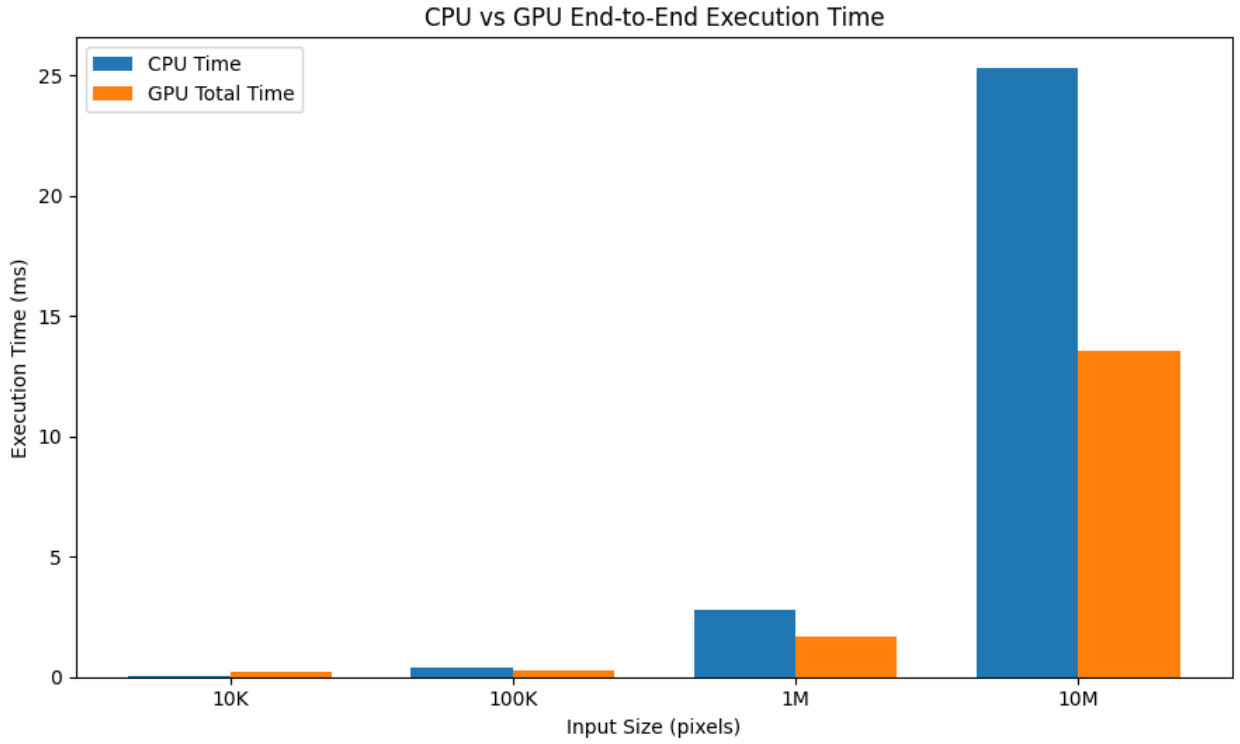


Figure 5. CPU versus GPU end-to-end execution time across different input sizes (10K, 100K, 1M, and 10M pixels).

The execution time of both implementations increases as the number of pixels grows. For the smallest input size of 10,000 pixels, the CPU completes the conversion faster because GPU memory-transfer and kernel-execution overheads are significant relative to the small workload. However, for 100,000 pixels and above, the CUDA implementation records lower total execution times than the CPU implementation. This demonstrates the benefit of parallel processing when a sufficiently large number of pixels can be processed concurrently.

10.3 Speedup Analysis

Figure 6 presents the end-to-end speedup achieved by the CUDA implementation compared with the sequential CPU implementation. Speedup is calculated as the ratio of CPU execution time to total GPU execution time.

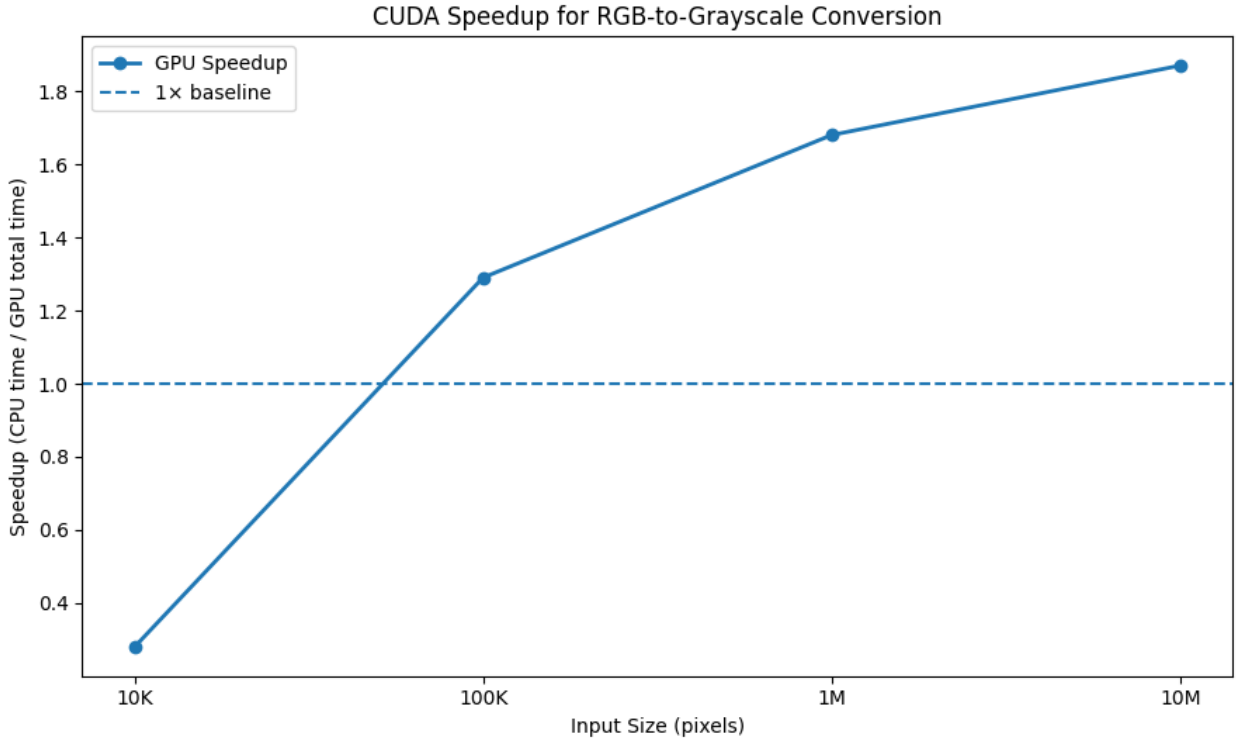


Figure 6. GPU speedup over CPU for end-to-end image blurring across different image sizes (dashed line indicates 1× baseline).

The measured speedup values were 0.28× for 10,000 pixels, 1.29× for 100,000 pixels, 1.68× for 1,000,000 pixels, and 1.87× for 10,000,000 pixels. A speedup greater than 1 indicates that the GPU implementation completed the measured workflow faster than the CPU implementation.

The results show that the performance advantage of CUDA increases with the tested workload size. The highest measured speedup of 1.87× was obtained for 10,000,000 pixels, demonstrating the effectiveness of GPU parallelism for larger workloads.

10.4 Effect of Problem Size

The experimental results show that input size significantly influences the performance of CUDA-based RGB-to-grayscale conversion. For small workloads, the time required for memory transfers and kernel execution can outweigh the computational benefits of GPU parallelism. This is reflected in the speedup of 0.28× for 10,000 pixels.

As the number of pixels increases, the GPU can distribute the conversion operations across more threads. The measured speedup rises to 1.29× at 100,000 pixels and 1.68× at 1,000,000 pixels. At 10,000,000 pixels, the speedup reaches 1.87×.

These results indicate that GPU acceleration becomes more beneficial as the workload grows. However, the measured improvement depends on both parallel computation and the overhead associated with transferring data between CPU and GPU memory.

10.5 Interpretation of Results

The end-to-end measurements demonstrate that CUDA acceleration depends on workload size and data-transfer overhead. For small inputs, GPU execution may be slower than CPU execution because kernel launch, synchronization, and memory transfers contribute significantly to total processing time.

For larger inputs, the GPU processes multiple pixels concurrently, reducing the overall execution time compared with the sequential CPU implementation. The increase in measured speedup from 0.28× at 10,000 pixels to 1.87× at 10,000,000 pixels illustrates this trend.

The results also confirm the correctness of the CUDA implementation, as no mismatches were observed between the CPU and GPU outputs. Overall, the experiment demonstrates that CUDA can improve RGB-to-grayscale conversion performance for sufficiently large workloads, while the sequential CPU approach can remain more efficient for small inputs.

11. Advantages of CUDA-BASED RGB-TO-GRAYSCALE CONVERSION

- **Highly parallel processing:** Thousands of CUDA threads can process different pixels concurrently, making RGB-to-grayscale conversion suitable for GPU execution.
- **Independent pixel computation:** Each grayscale output value is calculated independently from the corresponding pixel's Red, Green, and Blue components, requiring minimal communication between threads.
- **Effective for large workloads:** As the number of pixels increases, the GPU can utilize its parallel processing capability more effectively, reducing total execution time for sufficiently large inputs.
- **Straightforward CUDA implementation:** The conversion operation can be implemented using a compact CUDA kernel in which each thread processes one pixel and applies the weighted grayscale formula.
- **Applicable to different input sizes:** The same CUDA kernel can process different numbers of pixels by adjusting the grid dimensions according to the input size.

12. Limitations

- **Data transfer overhead:** Transferring RGB input data from host memory to GPU memory and retrieving the grayscale output can add significant time, particularly for small workloads.
- **Kernel execution overhead:** CUDA kernel launch and synchronization introduce additional processing costs that may reduce the benefits of GPU acceleration for lightweight tasks.
- **Limited device memory:** The maximum input size that can be processed depends on the available GPU memory and the memory required for input and output buffers.
- **Workload-dependent acceleration:** GPU processing does not necessarily provide better performance for every input size. In this experiment, the speedup was below 1× for 10,000 pixels, indicating that the CPU was faster for the smallest tested workload.
- **Memory bandwidth considerations:** RGB-to-grayscale conversion requires reading three colour components and writing one grayscale value for each pixel. Consequently, memory-transfer costs and memory bandwidth can influence overall performance.

13. Applications:

Although RGB-to-grayscale conversion is a relatively simple image-processing operation, it represents an important GPU computing pattern in which individual pixels are processed concurrently. Grayscale conversion is commonly used as a preprocessing step in larger image and video processing pipelines. Representative application areas that benefit from CUDA-accelerated grayscale conversion include:

- **Medical image processing:** Converting colour medical images into grayscale representations for analysis and visualization.
- **Computer vision:** Preparing images for edge detection, feature extraction, segmentation, and object recognition.
- **Photography and image enhancement:** Converting colour photographs into grayscale images for visual effects and image analysis.
- **Video processing:** Converting video frames into grayscale format for efficient downstream processing.
- **Surveillance systems:** Simplifying image data for selected monitoring and analysis tasks.
- **Graphics and multimedia applications:** Producing grayscale images for visual effects, image transformations, and multimedia processing workflows.

14. Conclusion:

This case study successfully implemented **RGB-to-grayscale conversion using NVIDIA CUDA**, where individual CUDA threads were assigned to process separate pixels of the input data. The implementation covered the complete GPU processing workflow, including host and device memory allocation, transfer of RGB data to the GPU, execution of the grayscale conversion kernel, device synchronization, retrieval of the processed output, and memory deallocation. The generated output was verified by comparing the CPU and GPU results, with zero mismatches observed across all tested input sizes.

The end-to-end performance evaluation demonstrated that the relative performance of CPU and GPU grayscale conversion depends on the workload size. For the smallest input of 10,000 pixels, the CPU completed the operation faster because GPU memory transfers and kernel-execution overhead contributed significantly to the total execution time. For larger inputs, the GPU utilized its parallel architecture more effectively. In the recorded benchmark, the CUDA implementation achieved a maximum end-to-end speedup of **1.87× at 10 million pixels**, compared with a speedup of 1.68× at 1 million pixels.

Overall, this experiment demonstrates how CUDA can be applied to a practical image-processing problem and highlights the importance of considering both computation and data-transfer overhead when evaluating GPU acceleration. RGB-to-grayscale conversion provides a useful example for understanding pixel-level parallelism, CUDA thread organization, memory management, kernel execution, correctness verification, and the performance trade-offs involved in GPU-based processing.